BitSemCom + GRAND

A guide to the repository: what is where, how it works, and how to run it

Safaa Salman

Thesis working document, November 2026, branch `grand-implementation`

What this document is

This document explains, in plain language, what is in the `BitSemCom` repository, how one image travels through the system, what every folder and script does, where the results are, and which conventions you must know before touching the code. It is written for students who join the project. The step-by-step commands are in the `README.md` at the repository root; this guide explains what those commands do. The numbers we measured are in the companion document *BitSemCom + GRAND: results so far*.

Contents

1 What this repository is **2**

2 How one image travels through the system **2**

3 Repository map **3**

4 Training: without and with GRAND **4**

4.1 The original recipe (`main.py`) . . . 4

4.2 Our trainer (`train_bitsemcom.py`) . . . 5

4.3 How GRAND is put inside the loop . . . 5

5 Evaluation **5**

5.1 Receivers . . . 5

5.2 Channel conditions . . . 6

5.3 The two sweep runners . . . 6

5.4 Metrics . . . 6

5.5 Figures . . . 6

6 The GRAND package **7**

7 Data, checkpoints and results **7**

8 Conventions you must know **8**

9 The most useful commands **8**

10 GRAND explained from scratch **8**

10.1 What a channel code is . . . 8

10.2 Decoding by guessing the noise . . . 9

10.3 ORBGRAND: ordering the guesses by reliability . . . 9

10.4 Where this lives in the code 10

11 How GRAND was put into BitSemCom **10**

12 Questions students ask **11**

13 Glossary **13**

1 What this repository is

The repository has two layers.

The original BitSemCom. Bit-level semantic image communication from Shanghai Jiao Tong University (arXiv 2510.26225). A Swin-transformer autoencoder (the "WITT" backbone) turns an image into a short sequence of bits, the bits are sent over a noisy channel as QPSK symbols, and a second Swin network rebuilds the image from what arrives. The authors call it *semantic* because the network decides what to put in the bits; nothing in the bitstream is a pixel value. This code is in `main.py`, `network/`, `data/`, `loss/` and `utils.py`, untouched.

Our addition: a channel code decoded by GRAND. The thesis question is whether an explicit error-correcting code, decoded with GRAND (Guessing Random Additive Noise Decoding), improves the image quality of such a system. We put a CRC code between the bit mapper and the semantic decoder, decode it with ORBGRAND (the soft-input version of GRAND), and measure image quality with and without it, on several channels, with several receivers, and also with GRAND inside the training loop. That code is in `grand/`, `experiments/`, `train_bitsemcom.py` and `matlab/`.

Branch `main` holds the original code only; branch `grand-implementation` holds everything described here.

2 How one image travels through the system

Figure 1 shows the path of one 256×256 image. The grey boxes are the original BitSemCom, the orange boxes are what we added. Each box is explained below.

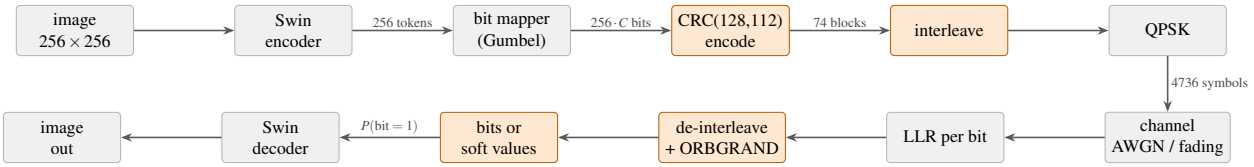


Figure 1: One image through the system for $C = 32$. Grey: the original BitSemCom. Orange: the coded link we added. Without the orange boxes the LLRs are turned into $P(\text{bit} = 1)$ directly and handed to the decoder; that is the original receiver.

Encoder and tokens.
The Swin encoder downsamples the image by 16 in each direction, so a 256×256 image becomes $16 \times 16 = 256$ *tokens*, each a feature vector.

Bit mapper and C.
For every token the bit mapper produces C bits with a Gumbel-softmax layer (`Resample_Layer` in `network/network.py`). C is the only size knob: $C = 16$, 32, 64 give 4096, 8192, 16384 bits per image. The layer is stochastic: sampling it twice flips about 5% of the bits, and the decoder was trained to live with that.

CRC code (ours).
The bits are cut into blocks of $k = 112$ and each block gets 16 parity bits from a CRC polynomial (Koopman notation `0x9eb2`), giving codewords of $n = 128$ bits, rate 0.875. The code is *systematic*: the codeword is the

112 data bits followed by the 16 parity bits. For $C = 32$ the 8192 bits are padded to 74 blocks (9472 coded bits), 16% more than uncoded.

Interleaver (ours).
A fixed pseudo-random permutation of all coded bits, known to both sides. In fading, a deep fade wipes out a whole token; after interleaving its bits are spread over many codewords and many tokens, which both the code and the semantic decoder handle far better.

QPSK and channel.
Two bits per symbol, Gray mapping, exactly the mapping of the original `QPSK_soft`. Channels: AWGN, or block Rayleigh fading with one complex gain per block of symbols (`fade8` = one gain per 8 symbols = one $C = 16$ token). Optional channel-state errors: the receiver may assume an SNR that is wrong by ± 3 dB, or use a noisy channel estimate.

LLR.
For every bit the receiver computes a log-likelihood ratio. Convention everywhere in this repository and in Duffy's code: **positive LLR means bit 0**. The original receiver converts it to $P(\text{bit} = 1) = \sigma(-\text{LLR})$ and gives that number to the decoder.

ORBGRAND (ours).
GRAND does not decode the code directly; it guesses the *noise*. It sorts the bits by reliability $|\text{LLR}|$, flips the least reliable bits first in a fixed order of patterns (the "landslide" pattern generator), and after each guess checks whether the result is a valid codeword ($Hc = 0$). The first valid codeword wins. If no codeword is found within the *guess budget* the block is *abandoned*. If a wrong codeword passes the check the block is *miscorrected*: the decoder then receives confidently wrong bits, which is the one way GRAND can hurt.

What the decoder receives (ours).
Decoded blocks: exact 0/1 values of their 112 data bits. Abandoned blocks: the plain soft values the uncoded receiver would have used (*soft fallback*; *hard fallback* gives 0/1 decisions instead). The padding is dropped and the $256 \cdot C$ values go to the Swin decoder, which cannot tell which receiver produced them.

3 Repository map

Path	What it is
`main.py`	the original training and test script (two stages: error-free channel, then noisy channel). Hard-coded dataset paths in its `config` class. Needs `lpips`, `thop`, `tensorboardX` and a CUDA GPU.
`network/network.py`	the `WITT` model: encoder, bit mapper (`Resample_Layer`), `QPSK_soft` (channel + LLR + $P(1)$), decoder.
`network/encoder.py`, `decoder.py`, `modules.py`	Swin blocks. `network/channel.py` and `network/settings.py` are not used by the bit path.
`data/datasets.py`, `loss/distortion.py`, `utils.py`	loaders, MS-SSIM, logging helpers of the original code.
`train_bitsemcom.py`	our trainer: fixed step budget, warmup + cosine learning rate, random SNR per batch, validation every N steps, best checkpoint, resume; `--grand` puts the coded link + ORBGRAND into the loop.
`grand/codes.py`	builds CRC and random linear codes: generator matrix $G = [I \mid P]$, parity-check matrix $H = [P^T \mid I]$, `encode`, `is_codeword`.
`grand/orbgrand.py`	ORBGRAND in pure numpy (basic and 1-line variants, landslide generator), port of Duffy's MATLAB. `ORBGRAND(H, max_queries).decode(llr)` returns (word, guesses, abandoned).
`grand/selftest.py`, `grand/validate_duffy.py`	self-test; reproduction of the block error rate curves shipped with GRAND-MATLAB.
`experiments/bitsemcom_io.py`	builds the model without importing `main.py`; loads checkpoints with a strict key check; encodes images to bits and decodes probabilities to images.

Path	What it is
`experiments/channel_coding.py`	the coded link (`CodedLink.transmit`): pad, encode, interleave, QPSK, channel with CSI errors, LLR, ORBGRAND, fallback, statistics. Also the uncoded QPSK channel in numpy.
`experiments/torch_channel.py`	the same uncoded QPSK channel in torch, differentiable, used in training.
`experiments/grand_layer.py`	`GrandChannel`: GRAND inside training. Forward value = the coded link's output; gradient = the uncoded soft channel's Jacobian (section 4).
`experiments/conditions.py`	the grammar of channel-condition names (section 5.2).
`experiments/metrics.py`	PSNR, MS-SSIM, LPIPS (AlexNet), CLIP ViT-B/32 cosine similarity.
`experiments/run_grand.py`	round-1 sweep runner: one model, several SNRs, all receivers; one JSONL row per (image, SNR, receiver).
`experiments/evaluate.py`	round-2 sweep runner: several models $\times$ conditions $\times$ SNRs $\times$ receivers, four metrics, optional MATLAB decoding, example images.
`experiments/plot_llr.py`	the LLR study: what the decoder and GRAND receive under each condition.
`experiments/plot_results.py`, `plot_training.py`, `plot_review_gains.py`, `plot_style.py`	all figures: quality curves, coding gains, GRAND block statistics, training curves, the paired-gain charts of the review, example strips; one shared style.
`experiments/summarize.py`	markdown tables from round-1 JSONLs.
`experiments/bit_importance.py`, `channel_importance.py`	round-1 diagnostics: flip a fraction of bits (random, most confident, least confident, whole tokens, per channel) and measure the PSNR drop.
`experiments/grand_verify.py`, `grand_backend.py`, `matlab/`	export of a test set of LLR blocks, bit-exact comparison with Duffy's MATLAB code, and a decoding backend that routes every ORBGRAND call through MATLAB or Octave.
`experiments/make_kodak_patches.py`	makes the 24 centre 256×256 crops of Kodak used everywhere.
`BitSemCom_Colab.ipynb`	the round-2 experiments as Colab cells (models, data and results on Google Drive).
`results/`	raw results, tables, figures, training logs (section 7).
`docs/`	the two markdown reports, their figures, and these LaTeX documents.
`Pretrain/`, `datasets/`	checkpoints and images; git-ignored, see the README for where to get them.

4 Training: without and with GRAND

4.1 The original recipe (`main.py`)

The authors train in two stages. Stage 1 trains the autoencoder with an error-free channel (the `WITT_BitSC_C*` checkpoints). Stage 2 starts from stage 1 and fine-tunes with `--pass_channel`: each epoch draws one integer SNR between `--snr_min` and `--snr_max`, sends the bits through `QPSK_soft` and trains the whole network on the reconstruction MSE. The learning rate is stepped down by `--decay` every `--step` epochs to `--min_lr`. The loop never ends by itself (ten million epochs); a checkpoint is written every 20 epochs to `history/BitSC_Prob/<timestamp>/models/`. Our starting checkpoints `Pretrain/BitSemCom_C{16,32,64}_stage2_ep*.model` are stage-2 models trained this way on Colab, with SNR in $[-3, 6]$ dB.

4.2 Our trainer (`train_bitsemcom.py`)

The supervisor asked for a benchmark "trained properly": a fixed budget of 30,000 steps, AdamW, 500 warmup steps then cosine decay from 10^{-4} to 10^{-6}, gradient clipping, SNR drawn per batch from the continuous range $[0, 10]$ dB, bf16 autocast for the Swin blocks, validation every 1000 steps on the 24 Kodak crops at 0, 3, 6 and 10 dB with fixed seeds, and the best checkpoint kept. One training step is:

1. image → encoder → bit mapper → bits (the Gumbel layer passes a gradient through its own straight-through trick);
2. channel: either the uncoded torch QPSK (`channel_uncoded`) or, when `--grand` is set, the GRAND channel described in the next subsection;
3. $P(1)$ → decoder → MSE on the 0–255 scale → backward → AdamW step.

4.3 How GRAND is put inside the loop

`GrandChannel.transmit` (`experiments/grand_layer.py`) moves the batch's bits to numpy, sends each image to a worker process, and the worker runs the very same `CodedLink.transmit` that evaluation uses (CRC encode, interleave, QPSK, channel, LLR, ORBGRAND with budget 2000, soft fallback). The $P(1)$ values come back as a tensor of the same shape the uncoded channel would return, so the decoder cannot tell which channel it is attached to.

Why the gradient needs a trick

Encoding, interleaving and ORBGRAND are not differentiable. The trainer therefore pushes the same bits through the uncoded torch channel a second time (fresh noise, same settings) and combines the two:

$$p_1 \;=\; \mathrm{stopgrad}(p_1^{\mathrm{GRAND}}) \;+\; \big(p_1^{\mathrm{soft}} - \mathrm{stopgrad}(p_1^{\mathrm{soft}})\big).$$

The value is exactly the GRAND output (the bracket is zero) while the gradient is the uncoded soft channel's Jacobian, so the encoder is trained by the same gradient path as in the benchmark run; only the forward signal the decoder sees differs (`--grand_grad soft`, the default; `identity` is plain straight-through).

The block statistics of every step (abandoned, miscorrected, guesses, decode time) go to `log.csv`. In the run we made, about half of the blocks were abandoned and 1.5% miscorrected, flat for all 30k steps, because with fixed-power QPSK the encoder has no handle on which blocks decode.

5 Evaluation

5.1 Receivers

Every receiver gets the same semantic bits and the same channel noise for a given (image, SNR, seed), so all comparisons are paired: the difference between two receivers on one image is a difference between two runs that share everything except the receiver.

Name	What the semantic decoder receives
`noiseless`	the exact bits: the ceiling for any channel code
`uncoded_soft`	$P(\text{bit} = 1)$ from the QPSK soft demodulator: the original BitSemCom receiver
`uncoded_soft_il`	the same, with the coded link's interleaver but no code: isolates the interleaving gain
`uncoded_hard`	hard 0/1 decisions
`coded_soft`	interleaver + CRC(128,112) + ORBGRAND, abandoned blocks fall back to the soft values
`coded_hard`	the same, abandoned blocks fall back to hard decisions
`*_ebn0`, `*_noil`	round 1 only: coded path at equal energy per semantic bit; coded path without interleaver

5.2 Channel conditions

Condition names (`experiments/conditions.py`) are tokens joined by underscores: `awgn` or `fade<N>` (N symbols per fading block; `fade1` is fast fading, `fade8` one gain per 8 symbols); `over<X>` / `under<X>`: the receiver assumes an SNR X dB too high / too low; `h<Y>`: channel-estimate error of variance Y (fading only); `rnd<X>`: SNR error drawn uniformly in $[-X, X]$ per transmission. The round-2 sweep used `awgn`, `awgn_over3`, `awgn_under3`, `fade8`, `fade8_h0.1` at E_s/N_0 from 0 to 12 dB in 2 dB steps.

5.3 The two sweep runners

`experiments/run_grand.py` (round 1) takes one model and one channel and writes one JSONL row per (image, SNR, receiver) with PSNR, MS-SSIM and, for coded receivers, the block statistics. `experiments/evaluate.py` (round 2) takes several models (`--ckpt name=file`), a list of condition names, SNRs and receivers, adds LPIPS and CLIP, can save example reconstructions (`--save_images`), and can decode through MATLAB (`--grand_backend`). The guess budget (`--budget`) matters: the main sweep uses 2000, the budget the GRAND model was trained with; a second sweep uses 100,000.

Each JSONL row has the keys `model`, `image`, `C`, `snr`, `condition`, `channel`, `fade_block`, `snr_err_db`, `h_err_var`, `path`, `channel_snr`, `payload_bits`, `psnr`, `msssim`, `lpips`, `clip` and, for coded receivers, `n`, `k`, `blocks`, `coded_bits`, `symbols`, `ber_channel`, `ber_after`, `bler_channel`, `bler_after`, `abandon_rate`, `miscorrect_rate`, `avg_queries`, `max_queries_seen`, `decode_time_s`.

5.4 Metrics

PSNR is the headline number (mean over the 24 images; differences below about 0.03 dB are noise). MS-SSIM and LPIPS (AlexNet; lower is better) are computed for every image. CLIP similarity (ViT-B/32 cosine similarity between the two images) was meant as a "semantic" metric but is too noisy on 24 images to separate receivers.

5.5 Figures

`plot_results.py` draws, per condition, the four metrics against SNR for every model and receiver (`quality_*.png`), the coding gain against the plain receiver (`gain_*.png`) and against the interleaver-only receiver (`gain_*_vs_il.png`), and the GRAND block statistics (`grand_*.png`); it also writes `summary.md`. `plot_training.py` draws the training curves from `log.csv/val.csv`. `plot_review_gains.py` draws the paired-gain bar charts of the review (room up to perfect bits against the coded gain, GRAND-trained minus benchmark, cost of imperfect CSI, budget 2000 against 100,000, example reconstruction strips).

6 The GRAND package

`grand/codes.py`: `make_crc(k, poly)` builds the systematic generator matrix G ($k \times n$) and parity-check matrix H ($(n-k) \times n$) from a CRC polynomial; `make_rlc` builds a random linear code; `encode(u, G)` is uG mod 2. `grand/orbgrand.py`: `ORBGRAND(H, max_queries, variant)` with `decode(llr)` and `decode_many`. Syndromes are packed into 64-bit integers and the noise patterns are cached, so the decoder runs at roughly two million guesses per second in pure numpy. `budget_mode='level'` reproduces the per-weight-level budget check of Duffy's MATLAB code, so a block may use up to about 15% more guesses than the nominal budget; `evaluate.py` and the trainer use it (round 2); the round-1 sweeps of `run_grand.py` stop exactly at the budget (`budget_mode='query'`).

The port was validated twice: `grand/validate_duffy.py` reproduces the block error rates and average guess counts of the `RESULTS/*.mat` files shipped with GRAND-MATLAB within Monte Carlo noise; and `experiments/grand_verify.py` exports 740 real LLR blocks for a bit-exact comparison with the unmodified MATLAB functions (`matlab/verify_port.m`).

Not yet verified bit for bit

The second check has not yet been run with a real MATLAB installation. The Octave attempt stopped on `cumsum(d,'reverse')`, which Octave lacks; `matlab/octave_compat/cumsum.m` is an untested stand-in. Until this is done, every number in the reports comes from the Python port.

7 Data, checkpoints and results

Data. Kodak (24 images) at `datasets/kodak_full`, and the centre 256×256 crops at `datasets/kodak_patches256` made by `experiments/make_kodak_patches.py`; every reported number is on those 24 crops. DIV2K train HR (800 images) is used for training only.

Checkpoints (`Pretrain/`, not in git): the authors' stage-1 models; the stage-2 models that all experiments start from, named `BitSemCom_C<C>_stage2_ep*.model` for $C = 16, 32, 64$; and the round-2 models `runs/C32_bench/best.model` and `runs/C32_grand/best.model`.

Results (`results/`): round-1 JSONLs `C16_*`, `C32_*`, `C64_*` with their markdown tables `summary_*.md` and the diagnostics `bit_importance_*.md`, `channel_importance_*.md`; round-2 sweeps `eval_C32.jsonl` (12,600 rows, budget 2000) and `eval_C32_b1e5.jsonl` (budget 100,000); training logs in `train_logs/` (`log.csv`, `val.csv` and the console output of the benchmark run, the GRAND run and the abandoned first GRAND attempt); all figures in `figures/`; the MATLAB test-set description in `grand_verify/`. Example reconstructions (`results/images_C32/`) and the test set itself (`testset.mat`) are git-ignored and live on the project Drive.

Reports (`docs/`): `grand_results.md` (round 1) and `grand_review.md` (round 2, with its figures in `docs/figures/grand_review/`), plus this guide and the results document in `docs/latex/`.

8 Conventions you must know

Conventions used everywhere in the code

- **LLR sign.** Positive LLR means bit 0, in BitSemCom, in Duffy's code and in ours. The decoder consumes $P(\text{bit}=1)=\sigma(-\text{LLR})$, never bits and never LLRs.
- **SNR.** Always E_s/N_0 per QPSK symbol in dB. The raw bit error rate is $Q(\sqrt{E_s/N_0})$: 16% at 0 dB, 5.7% at 4 dB, 2.3% at 6 dB, 0.6% at 8 dB. Duffy's BPSK curves at $E_b/N_0 = x$ correspond to this code's QPSK at $E_s/N_0 = x+3$ dB plus the rate term.
- **Equal** E_b/N_0 for a rate-R code means running the coded path at $E_s/N_0 + 10\log_{10}R$, i.e. -0.58 dB for 112/128.
- **Pairing.** The same (image, SNR, seed) gives the same bits and the same noise for every receiver; report paired differences with their standard error over the 24 images.
- **Budgets.** Evaluate a GRAND-trained model at the budget it was trained with (2000); a larger budget turns abandonments into miscorrections.
- **Code specs** are strings: `crc:0x9eb2:128:112` (CRC polynomial, n, k), `rlc:128:104:0` (random linear code, n, k, seed).

9 The most useful commands

All commands, with the data and checkpoint setup, are in the README. The ones used most often:

```
python grand/selftest.py
python experiments/run_grand.py --C 16 --snrs 2,4,5,6,7,8,10 --code crc:0x9eb2:128:112
python experiments/plot_llr.py --C 32 --limit 8 --snrs 0,4,8,12 --conditions awgn,awgn_over3,
    awgn_under3,fade8,fade8_h0.1,fade1 --budget 100000 --out results/figures
python train_bitsemcom.py --C 32 --train_dir datasets/DIV2K_train_HR --val_dir datasets/
    kodak_patches256 --steps 30000 --out runs/C32_bench
python train_bitsemcom.py --C 32 --train_dir datasets/DIV2K_train_HR --val_dir datasets/
    kodak_patches256 --steps 30000 --grand crc:0x9eb2:128:112 --grand_budget 2000 --out runs/
    C32_grand
python experiments/evaluate.py --C 32 --ckpt bench=runs/C32_bench/best.model --ckpt grand=runs/
    C32_grand/best.model --snrs 0,2,4,6,8,10,12 --conditions awgn,fade8 --paths noiseless,
    uncoded_soft,uncoded_soft_il,uncoded_hard,coded_soft --budget 2000 --out results/eval_C32.
    jsonl
python experiments/plot_results.py results/eval_C32.jsonl --out results/figures
python experiments/plot_review_gains.py results/eval_C32.jsonl --b1e5 results/eval_C32_b1e5.jsonl
     --out docs/figures/grand_review
```

10 GRAND explained from scratch

This section is for readers who have never met a channel code or GRAND. It uses the numbers of our own setup throughout: blocks of $k = 112$ data bits, codewords of $n = 128$ bits, 16 parity bits.

10.1 What a channel code is

The sender cuts the bit stream into blocks of 112 bits and appends 16 *parity bits* to each block. The parity bits are computed from the data bits by a fixed rule (ours: a CRC polynomial), so the receiver can recompute them. The 128-bit result is a *codeword*. Of all 2^{128} possible 128-bit words only 2^{112} are codewords, one in $2^{16} = 65{,}536$. The receiver checks membership with the parity-check matrix H: a word c is a codeword exactly when $Hc = 0 \pmod 2$;

the vector Hc is called the *syndrome*. Because our code is *systematic*, the 112 data bits sit unchanged at the front of the codeword: once a block is accepted, the receiver simply reads its first 112 bits.

Two numbers about our code matter later. The polynomial `0x9eb2` is one of the 16-bit polynomials recommended in Koopman's CRC tables: for data blocks of up to 135 bits (ours is 112) any two codewords differ in at least 6 positions, so no pattern of 5 or fewer flipped bits can turn one codeword into another.[1] And a random 128-bit word that is not a codeword passes the check with probability $1/65{,}536$.

10.2 Decoding by guessing the noise

A classical decoder is built for one family of codes and uses that family's structure (the trellis of a convolutional code, the graph of an LDPC code). GRAND (Guessing Random Additive Noise Decoding; Duffy, Li and Médard, 2019) does something simpler and more general. It does not search for the codeword; it searches for the *noise*. The received hard decisions y are the sent codeword c with some bits flipped, $y = c \oplus e$. GRAND lists candidate flip patterns e from the most likely to the least likely and asks, for each one, whether $y \oplus e$ is a codeword. The first yes wins and the answer is $c = y \oplus e$. Because the only thing it needs from the code is the membership test $Hc = 0$, the same decoder works for any linear code; you only hand it a different H. GRAND is practical when the code has few parity bits (ours has 16) and the channel errors are not too frequent, because that is when the right pattern is found after few guesses.

The order of the guesses is everything. With hard decisions alone every single flip is equally likely, every double flip is equally likely, and so on, so GRAND would try no flip, then the 128 single flips, then the 8,128 pairs: finding two errors can cost up to 8,257 guesses. With soft information it is far cheaper, because the wrong bits are almost always among the least reliable ones.

10.3 ORBGRAND: ordering the guesses by reliability

The demodulator gives every bit an LLR, and its magnitude $|\text{LLR}|$ says how sure the demodulator is. ORBGRAND (Ordered Reliability Bits GRAND; Duffy 2021; Duffy, An and Médard 2022) sorts the 128 bits from least to most reliable and gives them ranks $1, 2, \ldots, 128$. The *basic* variant gives every flip pattern a cost, the sum of the ranks of the bits it flips (the "logistic weight"), and tries the patterns in order of increasing cost: flip rank 1 (cost 1); rank 2 (cost 2); ranks 1 and 2, or rank 3 (cost 3); ranks 1 and 3, or rank 4 (cost 4); and so on. The Landslide algorithm (`landslide` in `grand/orbgrand.py`) generates all patterns of a given cost and number of flips directly, without listing everything first. The *1-line* variant, our default, first fits a straight line to the sorted reliabilities and charges each flipped bit an offset plus its rank instead of the rank alone; this follows the true likelihoods more closely when the noise is strong. In the self-test a block with two flipped bits is decoded in 5 guesses (basic) or 31 guesses (1-line) instead of thousands.

Two things follow from "only the order matters". Multiplying all LLRs by a constant does not change their ranking, so an SNR misestimate at the receiver changes nothing in ORBGRAND's decisions (it does change the soft values the semantic decoder sees). And there is no scale or threshold to tune: the decoder has a single parameter, the budget.

Budget, abandonment, miscorrection

ORBGRAND stops after a fixed number of guesses, the *budget*; a block that is not decoded by then is *abandoned*. Abandonment is an honest answer ("I could not decode this"), and our soft fallback makes it harmless: the semantic decoder then gets what it would have got without the code. The bad outcome is a *miscorrection*: a wrong word that happens to pass the check. Every wrong guess has about one chance in 65,536 of passing by accident, so when the true pattern is out of reach, about $2000/65{,}536 \approx 3\%$ of the blocks are miscorrected at a budget of 2000 and about 78% at a budget of 100,000. These rough estimates match what the sweeps measured at low SNR (2 to 3% and 67 to 81%). This is why a larger budget made the results worse: it does not make 16 parity bits stronger, it turns declared failures into hidden ones.

[1] `https://users.ece.cmu.edu/~koopman/crc/hd6.html`, row `0x9eb2`.

10.4 Where this lives in the code

`grand/codes.py` builds G and H from the polynomial (`make_crc`) or at random (`make_rlc`); `encode(u, G)` is uG mod 2. `grand/orbgrand.py` is the decoder. `decode(llr)` takes the hard decisions, computes their syndrome (that is guess number one; if it is zero the block is clean), sorts the bits by |LLR|, walks the schedule of costs, asks Landslide for the patterns of each cost, computes all their syndromes in one vectorised step from the pre-packed columns of H, and stops at the first pattern whose syndrome equals the received one. It returns the word, the number of guesses and the abandoned flag. Syndromes are packed into 64-bit integers, which is why $n-k$ must stay below 64.

11 How GRAND was put into BitSemCom

Where. Between the bit mapper and the semantic decoder, and nowhere else. The network is not modified: the encoder still produces $256 \cdot C$ bits and the decoder still receives $256 \cdot C$ numbers between 0 and 1. Everything we added lives in the channel in between.

At evaluation time (`CodedLink.transmit` in `experiments/channel_coding.py`) one image goes through these steps:

1. the 8192 bits ($C = 32$) are padded with 96 zeros to 74 blocks of 112, and each block is encoded to 128 bits (9472 coded bits, 4736 QPSK symbols instead of 4096);
2. the coded bits are shuffled by a fixed pseudo-random permutation, the interleaver (same seed on both sides), so that every codeword is spread over the whole image;
3. QPSK, the channel (AWGN or block fading, with the condition's CSI error), one LLR per bit;
4. the LLRs are un-shuffled and cut back into 74 blocks, and ORBGRAND decodes each block within the budget;
5. for a decoded block the first 112 bits are passed on as exact 0 or 1; for an abandoned block the 112 soft values $P(\text{bit} = 1)$ of its data bits are passed on instead (soft fallback); the 96 padding positions are dropped;
6. the semantic decoder reconstructs the image, and the block statistics (abandoned, miscorrected, guesses, time) go into the result row.

The receivers of section 5.1 are different exits from this chain: `uncoded_soft` skips steps 1, 2, 4 and 5; `uncoded_soft_il` keeps the interleaver but skips the code; `noiseless` skips the channel altogether.

Inside training (`GrandChannel` in `experiments/grand_layer.py`, switched on with `--grand`) the same `CodedLink.transmit` runs on every image of every batch, in a pool of worker processes (one image per job, 8 workers by default), with the budget of `--grand_budget` and the MATLAB-style budget accounting. The returned soft values are stacked into a tensor of the same shape the uncoded torch channel would produce and handed to the decoder. ORBGRAND is not the bottleneck: in the $C = 32$ run it took about 0.2 s of a 0.47 s training step.

The gradient trick, in words

Padding, encoding, interleaving and guessing are integer operations: there is no derivative to send back to the encoder. We therefore send the same bits a second time through the ordinary differentiable QPSK channel (with its own fresh noise) and build the tensor the decoder receives as "GRAND's values, plus (soft values minus a detached copy of the soft values)". The bracket is zero, so the decoder sees exactly GRAND's output; but autograd only knows about the soft channel, so the encoder receives exactly the gradient it receives in the benchmark run. The two runs therefore differ in one thing only: what the decoder is shown. That is deliberate, so that any difference between the two models can be attributed to the decoder learning, or not learning, to use GRAND's output.

What came out. The decoder did not learn to profit from GRAND: the GRAND-trained model is 0.1 to 0.24 dB worse than the benchmark with every receiver, and the code itself adds at most $+0.08$ dB on AWGN. In plain words: the semantic decoder was trained on noisy inputs and already shrugs off about 5% of scattered bit errors; a code can only fix scattered errors; and at the SNRs where 16 parity bits are enough there is less than 0.2 dB left to gain. The companion results document has the tables.

12 Questions students ask

Where do I start?
Run `python grand/selftest.py` (under a second, no GPU). Then run one tiny sweep, for example `python experiments/run_grand.py --C 16 --limit 2 --snrs 6 --out results/try.jsonl` (two images, one SNR, seven receivers: about 15 seconds on a laptop GPU, 14 rows), and read the rows it writes. Then read the results document. The README has every command; section 2 of this guide explains what they do.

Can I work without a GPU?
Only partly. Everything that runs the autoencoder needs a CUDA GPU, because the original Swin code places its attention masks on the GPU (`network/encoder.py`); that includes the sweeps, the LLR study and training. The GRAND decoder, its self-test and its validation against Duffy's curves, the summaries and all plotting from existing JSONL files run on the CPU.

Do I need the original main.py?
No, unless you want to reproduce the authors' own training or testing. Our scripts never import it; they build the model with `experiments/bitsemcom_io.py`, which also refuses a checkpoint whose keys do not match the model exactly. `main.py` needs extra packages (`lpips`, `thop`, `tensorboardX`) and hard-coded dataset paths.

Which checkpoint should I use?
The stage-2 checkpoints are the original models at $C = 16$, 32 and 64 and the starting point of everything. `runs/C32_bench/best.model` is the better $C = 32$ model from round 2, and `runs/C32_grand/best.model` is the one trained with GRAND in the loop. Without `--ckpt` the scripts take the newest `*C<C>_*.model` in `Pretrain/`.

What is C and which value should I use?
C is the number of bits per token; a 256×256 image has 256 tokens, so $256 \cdot C$ bits. Choosing C is choosing the bandwidth: doubling it adds about 1.1 dB of PSNR. Round 2 used $C = 32$.

What is a soft value and why does the decoder want it?
$P(\text{bit} = 1)$: 0.02 means "almost surely 0", 0.5 means "no idea". A faded or very noisy bit arrives as roughly 0.5 and the decoder treats it as unknown rather than wrong. A hard decision throws that away: at 0 dB it costs 1.7 dB on AWGN and 2.7 dB in fading. So keep soft values whenever you can, and read "coded beats hard decisions" with care: it mostly measures soft against hard, not GRAND.

Why does a positive LLR mean bit 0?
It is the convention of BitSemCom's `QPSK_soft` and of Duffy's code: $\text{LLR} = \log P(0)/P(1)$ and $P(1) = \sigma(-\text{LLR})$. Keep it if you add a channel; otherwise ORBGRAND flips the wrong bits and the decoder gets inverted probabilities.

Is an abandoned block a failure?
No. It means "not decoded, passed through". With soft fallback it costs nothing against the uncoded receiver, and it is logged (`abandon_rate`). A *miscorrected* block is the failure: wrong bits delivered with full confidence.

Why not just raise the budget?
Because 16 parity bits do not get stronger with more guesses; a larger budget mostly turns abandoned blocks into miscorrected ones (see the box in section 10). The 100,000-guess sweep lost 0.4 to 1.3 dB on AWGN and 0.5 to 2.2 dB in fading below the SNR where the code works.

Why a budget of 2000?
It is the budget the GRAND-in-the-loop model was trained with, so the main sweep evaluates it as it was trained. It is large enough that from 8 dB nearly every block decodes within about 10 guesses, and small enough to keep miscorrections at a few percent and the training step fast.

Why a CRC, and why this polynomial?
GRAND works with any linear code; a CRC is systematic, can be built for any block length from one polynomial, and 16 parity bits keep both the overhead (16%) and the guesswork small. `0x9eb2` comes from Koopman's tables (minimum distance 6 for our block length). Round 1 also tried a random linear code with 24 parity bits (`rlc:128:104:0`), which removed the miscorrections at a higher overhead, and a short CRC(64,48).

Why not an LDPC or polar code?
The thesis question is GRAND, and GRAND is at its best with short codes that have few parity bits. More parity bits also cost bandwidth, and in this system 16% of bandwidth is worth about 0.2 dB as semantic bits, more than any coding gain we measured.

Why is the SNR given as Es/N0 and not Eb/N0?

The channel carries QPSK symbols, so E_s/N_0 per symbol is what the simulation sets. For the uncoded system $E_b/N_0 = E_s/N_0 - 3$ dB; the coded path spends a further 0.58 dB per semantic bit (rate 112/128). Duffy's BPSK curves at $E_b/N_0 = x$ correspond to our $E_s/N_0 = x + 3 + 10\log_{10} R$. The `*_ebn0` receivers of round 1 run the coded path 0.58 dB lower so that both sides spend the same energy per semantic bit.

Why are the comparisons paired?

Every receiver gets the same bits and the same channel noise for a given image, SNR and seed, so the difference between two receivers on one image is caused by the receiver alone. Averaged over 24 images, that difference has a standard error of 0.01 to 0.05 dB, which is what makes gains of 0.05 dB readable at all. Never compare rows from different seeds or sweeps, and never trust a PSNR difference below about 0.03 dB.

Why can we not backpropagate through GRAND?

Encoding, interleaving and guessing are operations on integers; they have no derivative. The trainer uses the trick described in section 11: the forward value comes from GRAND, the gradient from a second pass through the soft channel (`--grand_grad soft`); the alternative `identity` is plain straight-through.

Why did GRAND in the loop make the model worse rather than better?

Three things were visible. The abandonment rate never moved during training, so the encoder cannot steer which blocks decode (the QPSK power and the code are fixed). The gap to the benchmark is the same with perfect bits as with the coded receiver, so the result is a worse autoencoder, not a model that specialised to GRAND. And the training loss stayed about 3% higher, which points at the surrogate gradient. The results document proposes the next test: freeze the encoder and fine-tune only the decoder.

Why 74 blocks and 96 zero bits?

$8192/112 = 73.1$, so 74 blocks are needed and the last one is filled with 96 zeros. The padding is dropped after decoding and is counted in the 16% overhead. For $C = 16$ it is 37 blocks, for $C = 64$ 147.

Does the semantic decoder know which bits GRAND decoded?

No. Its input is a vector of probabilities, and an "exact bit" is simply a probability of 0 or 1. The backbone has an SNR input that could carry such a hint, but in these checkpoints it was found to be inert. Passing an explicit flag is a possible extension.

What do fade8, fade16 and fade1 mean?

One Rayleigh gain per 8, 16 or 1 QPSK symbols. `fade8` is 16 bits: one $C = 16$ token or half a $C = 32$ token; `fade16` is one $C = 32$ token; `fade1` is fast fading. A deep fade on `fade8` erases a whole token, which is exactly what the semantic decoder handles worst and what the interleaver repairs.

How do I add a new code, channel condition or receiver?

A code is a string, `crc:<Koopman polynomial>:<n>:<k>` or `rlc:<n>:<k>:<seed>`, parsed by `make_code` in `experiments/channel_coding.py`; the polynomial degree must equal $n-k$ and $n-k$ must stay below 64. A channel condition is a name in the grammar of `experiments/conditions.py`, for example `fade16_h0.2_under2`. A receiver is a branch in the path loop of `experiments/evaluate.py` (round 2) or `experiments/run_grand.py` (round 1); copy the nearest existing branch and give it a new name.

How long does everything take?

The self-test under a second. The small sweeps minutes (the autoencoder on the GPU, ORBGRAND on the CPU); the full round-2 sweep is longer because of five conditions, seven SNRs, five receivers and three models, and LPIPS and CLIP download their weights on first use. Training is the expensive part: 30k steps took 5.8 h on an L4 and 3.9 h on an A100, with or without GRAND.

Can the Python decoder be trusted?

It reproduced the block error rates and guess counts published with Duffy's GRAND-MATLAB code (`grand/validate_duffy.py`), and the self-test checks the Landslide patterns by brute force. The bit-for-bit comparison on real LLR blocks (`experiments/grand_verify.py`, 740 blocks exported) still needs a MATLAB session; until then every number in the reports comes from the Python port.

What should I never do?

Compare numbers from different seeds or sweeps. Evaluate the GRAND-trained model at a budget it was not trained with. Mix E_s/N_0 and E_b/N_0. Feed hard bits to the decoder and call the loss a GRAND result. Flip the LLR sign.

13 Glossary

Semantic bits
The bits the bit mapper produces; the network decides what they mean.

Token
One of the 256 feature vectors the encoder produces for a 256×256 image; each becomes C bits.

LLR
Log-likelihood ratio of one bit: how sure the receiver is, and in which direction (positive = bit 0).

Soft value / $P(1)$
The probability that a bit is 1, computed from the LLR. "Soft" receivers keep it; "hard" receivers round it to 0 or 1.

Systematic code
A code whose codeword contains the data bits unchanged, followed by parity bits.

GRAND / ORBGRAND
Decoding by guessing the noise pattern, most likely first, and checking each guess against the code. ORBGRAND orders the guesses by the bit reliabilities.

Guess budget
The maximum number of guesses per block before the decoder gives up.

Abandoned block
A block where the budget ran out; passed on as soft values (harmless).

Miscorrected block
A block where a wrong codeword passed the check; passed on as confidently wrong bits (harmful).

Interleaver
A fixed shuffle of bit positions that spreads a burst of errors over many codewords and tokens.

Block fading
A channel whose gain is constant over a block of symbols and changes between blocks.

CSI Channel state information: what the receiver believes about the SNR and the channel gain.

Paired difference
The difference between two receivers on the same image with the same bits and noise, averaged over images.